Lesson: Creating an Express.js Project

Introduction

This lesson introduces you to building modern web applications using Node.js, TypeScript, and Express.js. You will learn how to set up your development environment from scratch, understand how each tool contributes to backend development, and create a basic but functional server. This foundation will prepare you to build more advanced features and scalable backend systems in the next lessons.

Learning Outcomes

By the end of this lesson, you will be able to:

1. Install and verify Node.js, TypeScript, and Express.js
2. Write and run a basic Node.js program
3. Use TypeScript to enforce types and prevent errors
4. Build a simple Express.js server and define routes
5. Organize your project into clear modules
6. Use environment variables for secure configuration

Building Modern Web Applications with Node.js, TypeScript, and Express.js

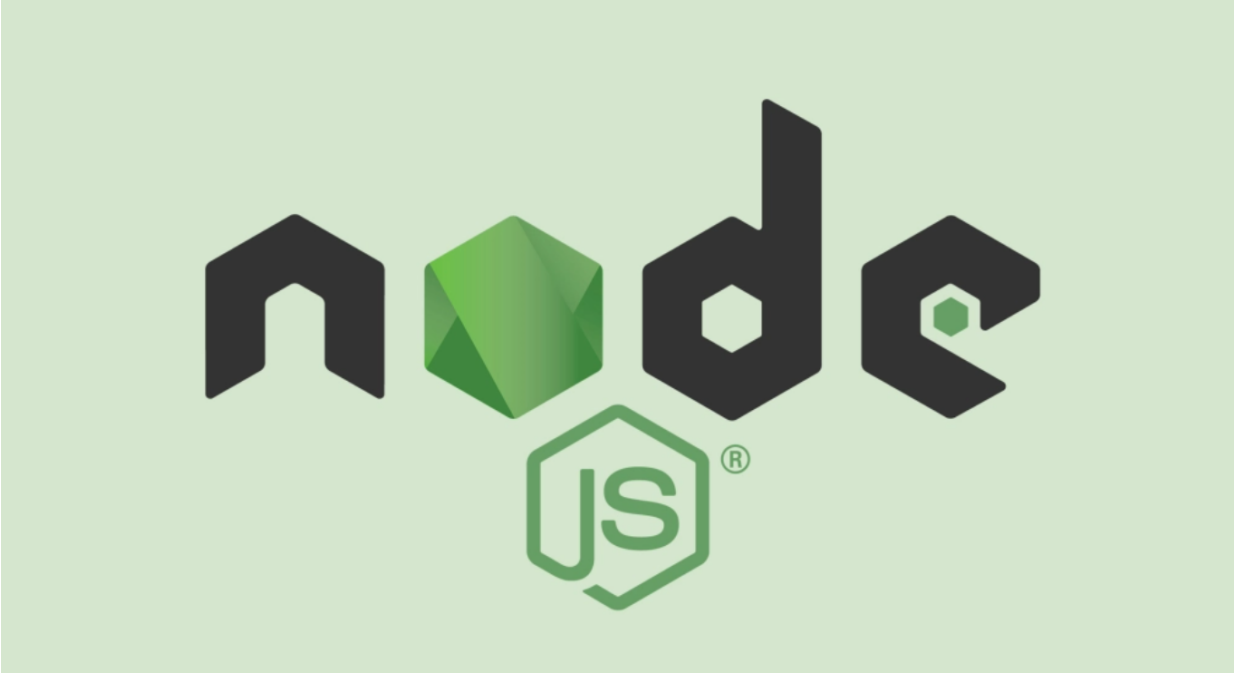
This is a walkthrough the process of setting up a modern web development environment using **Node.js**, **TypeScript**, and **Express.js**. These tools are foundational for building scalable, maintainable, and efficient web applications. We'll explain why each tool is essential, how to install them, and their roles in creating robust server-side applications.

Node.js: The Foundation of Server-Side JavaScript

Node.js is a game-changing runtime environment that allows developers to use JavaScript for server-side programming something that was traditionally dominated by languages like PHP, Java, or Python. With Node.js, JavaScript becomes a full-stack language, enabling developers to build scalable backend services with the same language they use on the frontend.

What is Node.js?

Node.js is a runtime environment that allows developers to run JavaScript on the server side. Traditionally, JavaScript was used only in web browsers to handle client-side interactions. However, Node.js extends JavaScript's capabilities, enabling developers to build server-side applications, APIs, and even full-stack applications.



Why Do We Need Node.js?

- **Unified Language**: With Node.js, you can use JavaScript for both frontend and backend development, creating a seamless workflow.
- **Rich Ecosystem**: Node.js provides access to a vast library of tools and packages via **npm (Node Package Manager)**, making it easy to integrate third-party functionalities.
- **Performance**: Node.js is lightweight, fast, and ideal for building scalable applications, especially those requiring real-time communication (e.g., chat apps).

How to Install Node.js

1. Visit the official Node.js website: https://nodejs.org.
2. Download the **LTS (Long-Term Support)** version, which is recommended for most users due to its stability.
3. Run the installer and follow the on-screen instructions.
4. Verify the installation by opening your terminal or command prompt and typing:

   ```
   node -v
   ```

   This command should display the installed version of Node.js (e.g., `v18.17.0`).

Testing Node.js

To confirm that Node.js is working correctly, create a simple JavaScript file (`test.js`) with the following code:

```
console.log("Hello, Node.js!");
```

Run the file using the command:

```
node test.js
```

Expected Output:

```
Hello, Node.js!
```

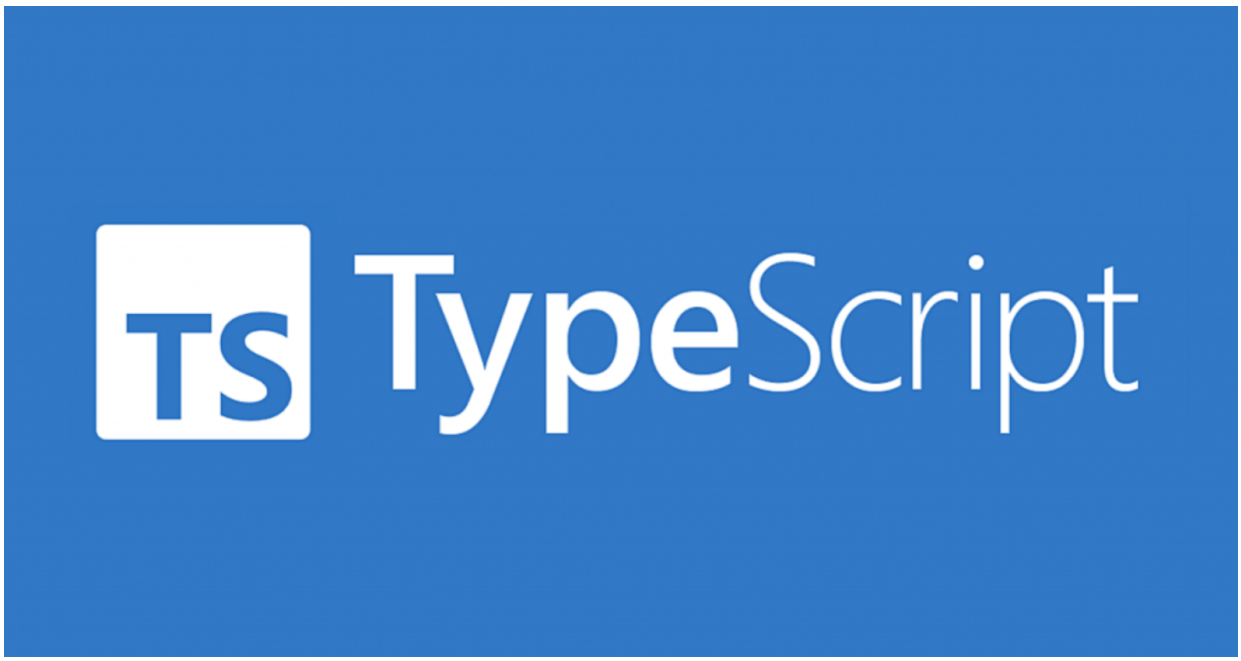
> NOTE: this is elaborated in the activities for this week

TypeScript: Adding Robustness to JavaScript

JavaScript is powerful and flexible—but with that flexibility comes the risk of bugs, especially in large or complex codebases. TypeScript steps in to solve this by adding static typing and modern tooling, making JavaScript development more reliable, scalable, and developer-friendly.

What is TypeScript?

TypeScript is a superset of JavaScript that adds static typing and other features to make JavaScript more robust and maintainable. It compiles down to plain JavaScript, ensuring compatibility with any environment that supports JavaScript.



Why Do We Need TypeScript?

- **Error Prevention**: TypeScript catches errors during development rather than at runtime, reducing bugs in production.
- **Code Quality**: Static typing improves readability and maintainability, especially in large projects with multiple contributors.

- **Advanced Features**: TypeScript supports modern JavaScript features and provides additional tools like interfaces, enums, and generics, which enhance code organization.

How to Install TypeScript

Once Node.js is installed, you can install TypeScript globally using npm:

```
npm install -g typescript
```

Verify the installation by checking the TypeScript version:

```
tsc -v
```

This should display something like `Version 5.2.2`.

Testing TypeScript

Create a simple TypeScript file (`hello.ts`) with the following code:

```
let message: string = "Hello, TypeScript!";
console.log(message);
```

Compile the TypeScript file into JavaScript using the TypeScript compiler:

```
tsc hello.ts
```

This will generate a `hello.js` file. Run the compiled JavaScript file using Node.js:

```
node hello.js
```

Expected Output:

```
Hello, TypeScript!
```

Explanation: The `string` type ensures that the variable `message` can only hold text values, preventing accidental assignment of numbers or other data types.

Express.js: Simplifying Web Application Development

Express.js is a fast, minimalistic, and flexible web application framework for Node.js. It provides a robust set of features for building APIs and web applications, making it one of the most widely used tools in the Node.js ecosystem.

What is Express.js?

Express.js is a minimal and flexible Node.js framework for building web applications and APIs. It simplifies routing, middleware integration, and request handling, making it easier to create robust server-side applications.

Why Do We Need Express.js?

- **Modular Structure**: Provides a clean and organized way to structure server-side code.
- **Simplified Tasks**: Streamlines common tasks like routing, handling HTTP requests, and serving static files.
- **Extensibility**: Backed by a large community and extensive plugin ecosystem, allowing you to extend its functionality as needed.

How to Install Express.js

1. Create a new project directory and initialize it with npm:

   ```
   mkdir my-app
   cd my-app
   npm init -y
   ```

   The `npm init -y` command creates a `package.json` file, which tracks your project's dependencies.

2. Install Express.js as a dependency:

```
npm install express
```

Creating a Simple Express Server

Create a file named `server.js` and add the following code:

```
const express = require('express');
const app = express();
const port = 3000;

// Define a route
app.get('/', (req, res) => {
  res.send('Hello, Express.js!');
});

// Start the server
app.listen(port, () => {
  console.log(`Server is running at http://localhost:${port}`);
});
```

Explanation of the Code

1. `const express = require('express');`: Imports the Express.js module.
2. `const app = express();`: Creates an instance of the Express application.
3. `app.get('/', ...);`: Defines a route for the root URL (`/`). When a GET request is made to `/`, the server responds with `"Hello, Express.js!"`.
4. `app.listen(port, ...);`: Starts the server on the specified port (3000). The callback logs a message to the console when the server starts.

Running the Server

Start the server using the following command:

```
node server.js
```

Expected Output in Terminal:

```
Server is running at http://localhost:3000
```

Open your browser and navigate to http://localhost:3000. You should see:

```
Hello, Express.js!
```

Organizing Your Project Structure

A well-organized project structure ensures maintainability, scalability, and readability. Here's an example of a clean project structure for a Node.js + TypeScript + Express.js application:

```
my-app/
├── src/
│   ├── controllers/      # Functions to handle requests
│   │   └── userController.ts
│   ├── routes/           # Define API routes
│   │   └── userRoutes.ts
│   ├── models/           # Database models (if using a database)
│   │   └── userModel.ts
│   ├── middleware/       # Custom middleware (e.g., authentication)
│   │   └── authMiddleware.ts
│   ├── services/         # Business logic and external API calls
│   │   └── userService.ts
│   ├── utils/            # Helper functions or utilities
│   │   └── logger.ts
│   ├── config/           # Configuration files (e.g., database,
environment variables)
│   │   └── envConfig.ts
│   ├── app.ts            # Main application setup
│   └── server.ts         # Entry point to start the server
├── .env                  # Environment variables
├── .gitignore            # Files to ignore in version control
├── package.json          # Project dependencies and scripts
├── tsconfig.json         # TypeScript configuration
└── README.md             # Documentation for the project
```

Best Practices for Scalable Applications

Here are some best practices for building scalable and maintainable applications with Node.js, TypeScript, and Express.js:

Best Practice	Description
Use TypeScript	Adds static typing to catch errors early and improve code readability. Compile with `tsc` or use tools like `ts-node`.
Organize Code into Modules	Split code into logical files (controllers, routes, services). Use dependency injection to improve testability and reduce clutter.
Manage Environment Variables	Store sensitive data in a `.env` file. Use `dotenv` to load them securely and add `.env` to `.gitignore` to avoid accidental version control.

Best Practice	Description
Implement Centralized Error Handling	Use middleware to manage errors. Ensure sensitive error information isn't exposed in production environments.
Use Logging Libraries	Incorporate libraries like `winston` or `morgan` for tracking app behavior and debugging.

Understanding Environment Variables and Importance

Environment variables are a fundamental concept in modern software development, especially for managing configuration in a secure, flexible, and environment-agnostic way. They allow developers to separate application logic from environment-specific settings, making applications more portable, secure, and maintainable.

What Are Environment Variables?

Environment variables are dynamic values that can influence how a program behaves when it runs. These variables are set outside the application code and provide essential information that affects the program's operation. They are widely used in various types of applications, including Single Page Applications (SPAs)**, Command-Line Interfaces (CLIs), and even shell scripts.

One of the key roles of environment variables is to configure an application for different environments, such as **development**, **testing**, **staging**, and **production**. By using environment variables, developers can ensure that the same codebase can run seamlessly across multiple environments without requiring changes to the core logic.

Why Do We Need Environment Variables?

The primary purpose of environment variables is to manage configuration settings for different environments. For example, you might want your application to connect to a local database during development but switch to a cloud-based database in production. Environment variables make this possible by allowing you to define these settings externally.

Additionally, environment variables are crucial for storing sensitive information securely. Common examples include:

- **API keys**: Used to authenticate with third-party services.
- **Auth tokens**: Used for user authentication and authorization.
- **Application configurations**: Settings like database URLs, ports, or other environment-specific parameters.

By keeping sensitive data out of your main codebase, environment variables enhance both the security and maintainability of your application. This separation ensures that sensitive information is not exposed in version control systems like Git and reduces the risk of accidental leaks.

How to Use Environment Variables in Your Application

To effectively use environment variables in your project, follow these steps:

Step 1: Install Dependencies

First, install the `dotenv` package, which helps manage environment variables in your application:

```
npm install dotenv
```

Step 2: Create a `.env` File

Next, create a `.env` file in the root directory of your project. This file will store your environment variables in a simple key-value format. For example:

```
PORT=3000
DATABASE_URL=mongodb://localhost:27017/mydb
JWT_SECRET=my_jwt_secret_key
```

Step 3: Load Environment Variables

In your application, load the environment variables using the `dotenv` package. For instance, in a TypeScript project, you can create a configuration file (`src/config/envConfig.ts`) to handle this:

```
import * as dotenv from 'dotenv';

// Load environment variables from the .env file
dotenv.config();

// Export the environment variables for use in the application
export const ENV = {
  PORT: process.env.PORT || 3000,
  DATABASE_URL: process.env.DATABASE_URL,
  JWT_SECRET: process.env.JWT_SECRET,
};
```

Step 4: Use Environment Variables in Your Application

You can now access these variables anywhere in your application. For example, in your `server.ts` file, you can start the server using the `PORT` variable:

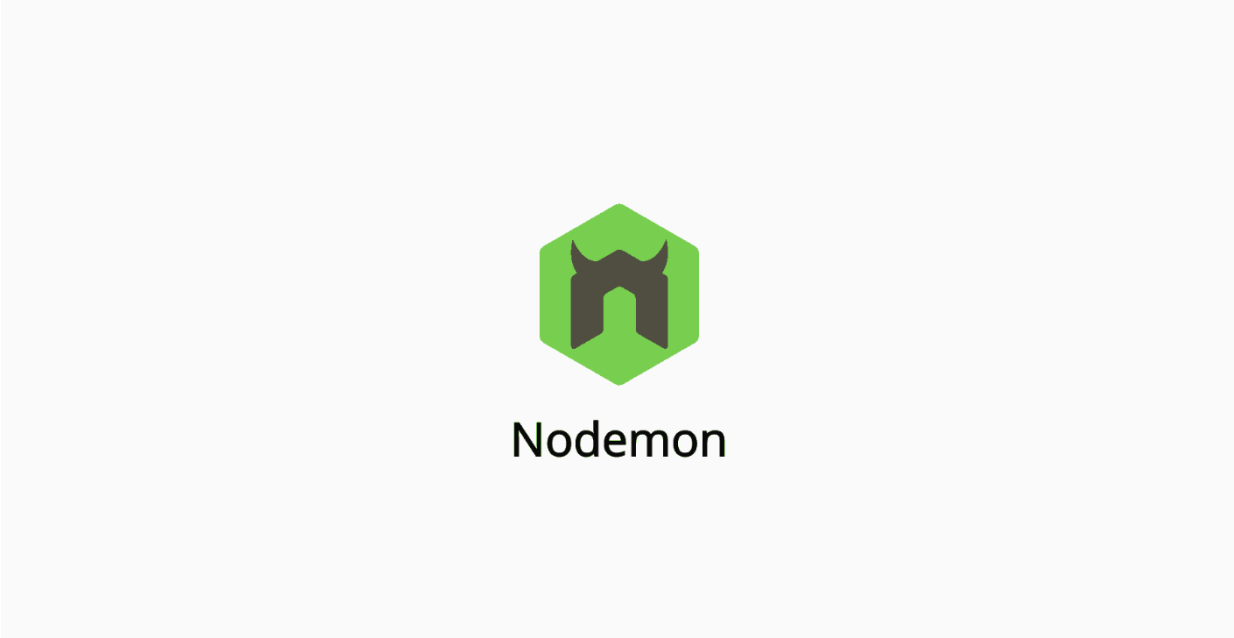
```
import express from 'express';
import { ENV } from './config/envConfig';

const app = express();

// Start the server on the specified port
app.listen(ENV.PORT, () => {
  console.log(`Server is running at http://localhost:${ENV.PORT}`);
});
```

This setup ensures that your application dynamically adapts to different environments without hardcoding sensitive or environment-specific details.

Enhancing Development Workflow with Nodemon

When working on TypeScript project, Nodemon is an invaluable tool that enables hot-reloading. This means your server automatically restarts whenever you make changes to your code, saving you time and effort during development.

Below, we'll explore how to integrate Nodemon into both CommonJS and ESM module systems.



Step 1: Install Required Dependencies

Start by installing **Nodemon** and **ts-node** as development dependencies:

```
npm install -D nodemon ts-node
```

- **Nodemon**: Monitors your files for changes and restarts the server automatically.
- **ts-node**: Compiles and executes TypeScript files on the fly, eliminating the need for pre-compilation.

Step 2: Configure Nodemon for CommonJS

If your project uses the **CommonJS** module system (i.e., you do **not** have `"type": "module"` in your `package.json`), follow these steps:

1. Create a `nodemon.json` File

Create a `nodemon.json` file in the root directory of your project and add the following configuration:

```
{
  "watch": ["src"],
  "ext": "ts",
  "exec": "ts-node ./src/index.ts"
}
```

- **"watch"**: Specifies the directories or files to monitor for changes (e.g., `src`).
- **"ext"**: Defines the file extensions to watch (e.g., `.ts`).
- **"exec"**: Specifies the command to execute when changes are detected (e.g., `ts-node ./src/index.ts`).

2. Add a Script to `package.json`

Add the following script to the `scripts` section of your `package.json`:

```
"scripts": {
  "dev": "nodemon --watch 'src/**/*.ts' --exec ts-node src/index.ts"
}
```

3. Start the Server

Run the following command to start your server with hot-reloading:

```
npm run dev
```

With this setup, **Nodemon** will automatically restart your server whenever you modify any `.ts` file in the `src` directory.

Step 3: Configure Nodemon for ESM

If your project uses the **ESM** module system (i.e., you have `"type": "module"` in your `package.json`), follow these steps:

1. Update `nodemon.json`

Modify your `nodemon.json` file to include the following configuration:

```
{
  "watch": ["src"],
  "ext": "ts",
  "execMap": {
    "ts": "node --no-warnings=ExperimentalWarning --loader ts-node/esm"
  }
}
```

- **"execMap"**: Maps `.ts` files to the appropriate execution command for ESM.
- **"node --no-warnings=ExperimentalWarning --loader ts-node/esm"**: Ensures compatibility with ESM by using the `ts-node/esm` loader.

2. Add a Script to `package.json`

Add the following script to the `scripts` section of your `package.json`:

```
"scripts": {
  "dev": "nodemon src/index.ts"
}
```

3. Start the Server

Run the following command to start your server with hot-reloading:

```
npm run dev
```

Just like with CommonJS, **Nodemon** will monitor your `.ts` files and restart the server whenever changes are detected.

Conclusion

By the end of this lesson, you will be able to write type-safe, scalable, and organized backend code using TypeScript and Express on top of Node.js. You will understand how to structure your project, run and test your server, and prepare it for future improvements. This knowledge will help you move forward with confidence as you develop more complex applications.